SOA and Web Services/Development of RESTful services

Lecture 21 & 22

Mahhek Tahir

Motivation

How does your React app get data?

How does Uber/Careem connect riders and drivers?

How do different systems talk to each other?

Traditional applications

One big application (Monolithic)

Everything tightly connected

Hard to scale

Hard to maintain

What is SOA?

SOA = Service-Oriented Architecture

- System divided into small independent services
- Each service performs a specific function
- Services communicate with each other

Campus Ride Sharing System

User Service

Ride Service

Booking Service

Benefits of SOA

- Easy to maintain
- Scalable
- Reusable services
- Independent development

What is a Service?

A **service** is:

- A small functional unit
- Performs a specific task
- Can be accessed over a network

For example: ‘Get all rides’ is a service.

What are Web Services?

- A way for systems to communicate over the internet
- Uses HTTP protocol
- Exchanges data between client and server

Web services are how services actually talk to each other.

API VS Web Service

An API is just a way for two pieces of software to communicate.
A Web Service is an API that works over the internet (HTTP).

Types of Web Services

SOAP
REST

SOAP (Simple Object Access Protocol)

SOAP is a protocol used for exchanging structured information between systems over a network.

- It is a **Web Service protocol**
- Uses **XML** format only (no JSON)
- Works over **HTTP**, SMTP, etc.

SOAP is an older, strict way of building web services, mostly used in enterprise systems like banking.

Soap Sample Request

```
<Envelope xmlns="http://schemas.xmlsoap.org/soap/envelope/">

  <Header>
    <AuthToken>
      <Token>abc123xyz456</Token>
    </AuthToken>
  </Header>

  <Body>
    <GetRideDetails>
      <RideId>123</RideId>
    </GetRideDetails>
  </Body>

</Envelope>
```

```
const soapBody = `
<Envelope xmlns="http://schemas.xmlsoap.org/soap/envelope/">
  <Header>
    <AuthToken>
      <Token>abc123xyz</Token>
    </AuthToken>
  </Header>
  <Body>
    <GetRideDetails>
      <RideId>123</RideId>
    </GetRideDetails>
  </Body>
</Envelope>
`;

fetch("https://example.com/soap-api", {
  method: "POST",
  headers: {
    "Content-Type": "text/xml"
  },
  body: soapBody
})
.then(res => res.text()) // SOAP returns XML
.then(data => console.log(data));
```

Node

npm install xml2js

```
const express = require("express");
const xml2js = require("xml2js");

const app = express();

// Middleware to get raw XML
app.use(express.text({ type: "text/xml" }));

app.post("/soap", (req, res) => {
  const xml = req.body;

  xml2js.parseString(xml, (err, result) => {
    if (err) {
      return res.status(400).send("Invalid XML");
    }

    console.log(result);

    // Access data
    const rideId =
      result.Envelope.Body[0].GetRideDetails[0].RideId[0];

    console.log("Ride ID:", rideId);

    res.send("Processed");
  });
});
```

Representational State Transfer

- A **design style** for building web services
- Uses standard **HTTP methods** (GET, POST, PUT, DELETE)
- Works with **resources** (e.g., `/users`, `/rides`)
- Data is usually exchanged in **JSON format**
- **Stateless** means each request is independent
- Simple, lightweight, and widely used in modern applications

REST is not a technology or tool, it is a set of rules for designing APIs. It tells us how clients and servers should communicate in a simple and structured way.

Sample Request

```
POST /rides HTTP/1.1
Host: example.com
Content-Type: application/json

{
  "pickup": "FAST",
  "destination": "DHA",
  "seats": 3
}
```

Creating a new ride
Data sent in **JSON**

With Token

```
GET /my-bookings HTTP/1.1
Host: example.com
Authorization: Bearer abc123xyz
```

With Query Parameters

```
GET /rides?destination=Lahore&seats=2 HTTP/1.1
Host: example.com
```

React

fetch("/rides?destination=Lahore")

In MERN Stack

Node/Express backend IS a web service

How do two Web Services communicate?

Two web services communicate by sending **HTTP requests to each other's APIs**

Service A → sends request → Service B → sends response back

Web services communicate exactly like frontend and backend, using HTTP requests.

One web service becomes a client for another web service.

Routes

Method	Endpoint	Type	Description
GET	/rides	Resource	Get all rides
POST	/rides	Resource	Create a new ride
GET	/rides/:id	Resource	Get details of a specific ride
DELETE	/rides/:id	Resource	Delete a ride

RESTful Routes with Nested Resources

Method	Endpoint	Type	Description
POST	/rides/:id/bookings	Nested Resource	Book a seat in a ride
GET	/rides/:id/bookings	Nested Resource	Get all bookings for a ride

Method	Endpoint	Type	Description
GET	/bookings	Resource	Get bookings of logged-in user
GET	/users/:id/bookings	Nested Resource	Get bookings of a specific user

Shallow Routing in RESTful APIs

What is Shallow Routing?

Shallow routing is a technique in **RESTful API design** where:

- **Nested routes are used only when necessary**
- **Avoid deep URL nesting for actions on a single resource**

It keeps URLs **short, readable, and maintainable**

Problem with Deep Nesting

Example of fully nested routes:

```
/rides/:ride_id/bookings/:id
/rides/:ride_id/bookings/:id/edit
/rides/:ride_id/bookings/:id/delete
```

Too long

Redundant (`booking id` is already unique)

Hard to manage

Solution: Shallow Routing

Use nesting **only when context is required**

```
GET    /rides/:ride_id/bookings      → list bookings for a ride
POST   /rides/:ride_id/bookings      → create booking

GET    /bookings/:id                 → show booking
PUT    /bookings/:id                 → update booking
DELETE /bookings/:id                 → delete booking
```

STATUS CODES

Code	Meaning
200	OK
201	Created
400	Bad Request
401	Unauthorized
404	Not Found
500	Server Error

Consistency in API design

Bad:

```
/getRides
/addRide
```

Good:

```
/rides
```

Naming convention of routes

REST uses **nouns (resources), not verbs**

`/rides/:id/book` → “book” is a **verb (action)**

`/rides/:id/bookings` → “bookings” is a **resource (noun)**

Data Formats in REST APIs

- **JSON** *(Most common)*
- **XML**
- **Plain Text**
- **HTML**
- **CSV**

Content-Type: application/json

Content-Type: application/xml

Content-Type: text/plain

```
app.get("/ride", (req, res) => {
  res.set("Content-Type", "text/plain");

  res.send("Pickup: FAST, Destination: DHA, Seats: 3");
});
```

html

```
const htmlData = `
  <div>
    <h1>Ride Info</h1>
    <p>Pickup: FAST</p>
    <p>Destination: DHA</p>
  </div>
`;

fetch("/ride-html", {
  method: "POST",
  headers: {
    "Content-Type": "text/html"
  },
  body: htmlData
})
.then(res => res.text())
.then(data => console.log(data));
```